

SmartParking

Documentazione del progetto – Testing e Verifica del Software

Francesco Tomasoni

A.A. 2025/2026

Contents

1	Introduzione e requisiti	3
1.1	Descrizione del sistema	3
1.2	Requisiti funzionali	3
1.3	Requisiti non funzionali	4
1.4	Macchina a stati	4
1.5	Glossario	4
2	Modello Asmeta	5
2.1	Il modello (<code>asmeta/src/SmartParking.asm</code>)	5
2.1.1	Il modello semplificato per il model checking	6
2.1.2	Simulazione manuale	6
2.2	Scenari Avalla	7
2.3	AsmetaV – validazione con copertura (Vc)	8
2.4	Model checking (<code>asmeta/src/SmartParking_MC.asm</code>)	9
2.4.1	Model Advisor (opzionale)	10
2.4.2	ATGT – generazione automatica di scenari (opzionale)	11
3	Implementazione Java	14
3.1	Struttura del progetto Maven	14
3.2	La macchina a stati	14
3.3	Interfaccia utente (Vaadin)	14
3.4	Test end-to-end con Selenium	16
4	Testing Java	17
4.1	Struttura dei test	17
4.2	MC/DC	18
4.3	Combinatorial testing (pairwise)	18
4.4	Mockito	19
4.5	Model Based Testing	19
4.6	Esecuzione e copertura	19
4.7	Mutation testing con PIT	20
5	JML e verifica statica (ESC)	23
5.1	Invarianti	23
5.2	Costruttori	23
5.3	Metodo <code>assegnaPosto</code>	24
5.4	Metodo <code>liberaPosto</code>	24
5.5	Metodi getter	25
5.6	Verifica dei contratti con OpenJML (ESC)	25

5.7	Verifica dinamica: i contratti a runtime (RAC)	26
6	Ispezione del codice e analisi statica	27
6.1	Ispezione con la checklist	27
6.2	Seconda lettura con un LLM	29
6.3	Analisi automatica con SpotBugs	30
6.4	Bilancio	31
7	Continuous Integration	31
8	Conclusioni	32
8.1	Riepilogo delle evidenze	32
8.2	Cosa mi porto a casa	33

1 Introduzione e requisiti

1.1 Descrizione del sistema

SmartParking è un parcheggio automatico. Ci sono due sbarre, una in ingresso e una in uscita, dei sensori che si accorgono quando arriva un'auto e un totem che legge che tipo di utente è. I posti sono di due tipi: standard e riservati ai disabili. Nel modello ne ho messo uno solo per tipo, altrimenti il model checker dopo esplode.

Gli utenti sono tre:

- **STD**: l'utente occasionale, è l'unico che paga;
- **DISABILE**: ha il permesso, non paga e ha la precedenza sui posti riservati;
- **ABBONATO**: ha l'abbonamento mensile e non paga.

Quando arriva un'auto il sistema la rileva, guarda che utente è e controlla se c'è un posto adatto. Se c'è alza la sbarra, se non c'è nega l'accesso.

Il caso un po' più interessante è quello del disabile. Lui i posti riservati li prende per primo, ma se sono finiti il sistema non lo respinge: gli dà un posto standard, se ce n'è uno libero. Questo è il *fallback*, e torna spesso nel resto del documento perché è il ramo che ha richiesto più attenzione un po' ovunque. Solo se sono finiti anche i posti standard viene negato l'accesso anche a lui.

In uscita paga solo l'utente STD, gli altri due escono e basta.

Il giro è questo: prima il comportamento come macchina a stati in Asmeta, poi la validazione con gli scenari Avalla e il model checking, e infine l'implementazione in Java con i contratti JML sul nucleo.

1.2 Requisiti funzionali

ID	Descrizione	Priorità
RF1	Rilevare l'arrivo di un veicolo in ingresso (sens_in) e passare da IDLE a CHK_IN.	Alta
RF2	Identificare il tipo di utente (STD, DISABILE, ABBONATO) e passare allo stato VER.	Alta
RF3	Per STD/ABBONATO, assegnare un posto standard se posti_std > 0 e aprire la sbarra (INGR).	Alta
RF4	Per DISABILE, assegnare in via prioritaria un posto disabile se posti_dis > 0.	Alta
RF5	Se per un DISABILE i posti disabili sono esauriti, tentare il fallback su posto standard.	Alta
RF6	Se nessun posto è disponibile (inclusi i fallback), negare l'accesso (NEG).	Alta
RF7	Da NEG, tornare in IDLE solo dopo che il veicolo si è allontanato (auto_via).	Media
RF8	Al transito in ingresso, decrementare il contatore del posto assegnato e tornare in IDLE.	Alta
RF9	Rilevare l'arrivo di un veicolo in uscita (sens_out) e passare a CHK_OUT.	Alta
RF10	In uscita, per STD richiedere il pagamento (TARIF) prima della sbarra.	Alta

ID	Descrizione	Priorità
RF11	In uscita, per DISABILE/ABBONATO saltare il pagamento e andare direttamente a USC.	Alta
RF12	Restare in TARIF finché il pagamento non è confermato.	Alta
RF13	Al transito in uscita, incrementare il contatore, azzerare la memoria del posto e tornare in IDLE.	Alta
RF14	Mantenere in memoria il tipo di posto occupato dall'ingresso fino all'uscita.	Media

1.3 Requisiti non funzionali

ID	Descrizione
RNF1	Sicurezza dei contatori: <code>posti_std</code> e <code>posti_dis</code> non devono mai essere negativi né superare la capacità iniziale, in ogni stato raggiungibile (verificato via model checking).
RNF2	Determinismo: a parità di stato e input monitorati, il sistema produce sempre lo stesso stato successivo.
RNF3	Assenza di stati bloccanti: da ogni stato transitorio esiste sempre un cammino che riporta il sistema in IDLE.

1.4 Macchina a stati

Qui sotto ci sono gli 8 stati del modello e le transizioni che li collegano. Il giro sopra è quello di ingresso, quello sotto l'uscita, e da IDLE si parte sempre.

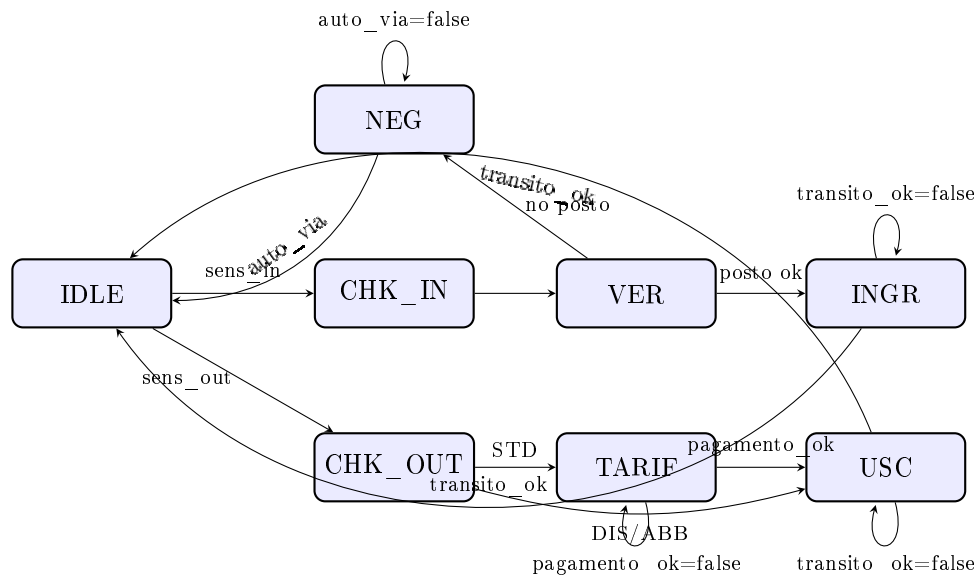


Figure 1: Diagramma della macchina a stati SmartParking (8 stati).

1.5 Glossario

Termine		Significato
Sensore ingresso/uscita	di	Rileva la presenza di un veicolo davanti a una sbarra.
Sbarra		Barriera fisica aperta solo in INGR/USC quando <code>transito_ok=true</code> .
Transito		Il veicolo è passato fisicamente sotto la sbarra aperta.
Tipo utente		Categoria del veicolo: STD, DISABILE o ABBONATO.
Posto standard / disabile		Le due categorie di posto auto.
Fallback		Politica per cui un DISABILE senza posti riservati riceve un posto standard.
Tariffa (TARIF)		Fase di pagamento, obbligatoria solo per STD in uscita.

2 Modello Asmeta

2.1 Il modello (asmeta/src/SmartParking.asm)

Partiamo dalla signature. I domini enumerativi sono tre: `StatoSistema` con gli 8 stati, `TipoUtente` con i tre tipi di utente e `TipoPosto` con i due tipi di posto più il valore `NESSUNO`, che vale quando non c'è nessun posto occupato.

Poi ci sono sei funzioni monitorate, che sono quelle che arrivano dall'esterno e che il modello subisce: `sens_in` e `sens_out` per i due sensori, `transito_ok` per dire che l'auto è passata sotto la sbarra, `auto_via` per dire che se n'è andata dopo un rifiuto, `utente_rilevato` per il tipo di utente letto dal totem e `pagamento_ok` per l'esito del pagamento.

Le funzioni controllate, cioè la memoria del sistema, sono quattro: `stato`, i due contatori `posti_std` e `posti_dis`, e `posto_assegnato`, che si ricorda che tipo di posto ha preso l'auto entrata. Quest'ultima serve perché in uscita bisogna sapere quale contatore rimettere a posto.

Le funzioni derivate sono due. La prima è `assegnabile`, ed è la più interessante delle due perché è n-aria: prende due argomenti, un utente e un posto, e dice se quel posto si può dare a quell'utente.

Listing 1: Funzione derivata assegnabile

```

1 derived assegnabile: Prod(TipoUtente, TipoPosto) -> Boolean
2
3 function assegnabile($u in TipoUtente, $p in TipoPosto) =
4     ($p = POSTO_STD and posti_std > 0) or
5     ($p = POSTO_DIS and $u = DISABILE and posti_dis > 0)

```

Letta a parole: un posto standard lo do a chiunque, basta che ce ne sia uno libero; un posto disabile lo do solo a un disabile, e solo se ce n'è uno libero. Il vantaggio di averla messa qui è che tutta la politica di assegnazione sta in due righe, in un punto solo. Essendo derivata non occupa memoria, viene ricalcolata a ogni stato partendo dai contatori.

La seconda derivata è `parcheggioPieno`, che serve soprattutto a usare il `forall`:

Listing 2: Funzione derivata parcheggioPieno (uso di forall)

```

1 function parcheggioPieno =
2     (forall $u in TipoUtente with
3         (not assegnabile($u, POSTO_STD) and not assegnabile($u, POSTO_DIS)))

```

Diventa vera quando *nessun* tipo di utente può più entrare, né su un posto standard né su uno disabili. La controllo nello scenario `test_pieno_totale.avalla`: all'inizio è falsa perché c'è ancora posto, alla fine diventa vera.

Veniamo alle regole. Quella che conta davvero è `r_gestione_VER`, dove si decide chi entra e chi no. C'è una regola `let` per non ripetere `utente_rilevato` ogni volta, e dentro la derivata di prima:

Listing 3: Regola `r_gestione_VER` (assegnazione posto, con `let` e derivata)

```

1 rule r_gestione_VER =
2   let ($u = utente_rilevato) in
3     if assegnabile($u, POSTO_DIS) then
4       par
5         stato := INGR
6         posto_assegnato := POSTO_DIS
7       endpar
8     else
9       if assegnabile($u, POSTO_STD) then
10        par
11          stato := INGR
12          posto_assegnato := POSTO_STD
13        endpar
14      else
15        stato := NEG
16      endif
17    endif
18  endlet

```

L'ordine dei due `if` non è casuale. Si prova prima con il posto disabili, che è vero solo se l'utente è disabili e c'è un riservato libero. Se quel test fallisce si passa al posto standard, e questo secondo ramo copre due casi diversi in una volta: l'ingresso normale di uno STD o di un abbonato, e il fallback del disabili quando i riservati sono finiti. Se falliscono tutti e due il sistema va in NEG.

Le altre regole sono più semplici. `r_gestione_IDLE` guarda quale dei due sensori si è acceso e va in `CHK_IN` o in `CHK_OUT`. `r_gestione_CHK_IN` passa subito a VER, serve solo a rappresentare il tempo di lettura del totem. `r_gestione_INGR` aspetta il transito e poi scala il contatore giusto, guardando `posto_assegnato`. `r_gestione_CHK_OUT` manda in TARIF solo se l'utente è STD, altrimenti va dritto in USC. `r_gestione_TARIF` resta ferma finché il pagamento non va a buon fine. `r_gestione_USC` rimette a posto il contatore e azzerà `posto_assegnato`.

2.1.1 Il modello semplificato per il model checking

Il modello principale non è utilizzabile per il model checking, quindi ne esiste una copia semplificata in `SmartParking_MC.asm`. Le differenze sono due.

La prima è che le funzioni derivate non ci sono e gli `if` sono riscritti in forma “appiattita”. Si comportano allo stesso modo, ma la traduzione verso NuSMV va liscia.

La seconda riguarda i contatori: nel modello principale sono `Integer`, e AsmetaSMV i domini infiniti non li accetta, quindi qui stanno su un dominio finito `Posti = {0,1,2}`. Il 2 è lasciato apposta anche se non serve: con `{0,1}` le proprietà `ag(posti_std <= 1)` e `ag(posti_dis <= 1)` sarebbero state vere per forza, solo per come è fatto il tipo, e non avrebbero verificato niente. Lasciando dentro il 2 il model checker deve davvero dimostrare che quel valore non viene mai raggiunto.

2.1.2 Simulazione manuale

Appena scritto il modello, la prima cosa è stata aprirlo nel simulatore e fare qualche step a mano, per vedere se si comportava come previsto prima di passare agli scenari.

	Type	Functions	^	State 0	State 1	State 2	State 3	State 4	State 5	
	M	sens_in		false	true	true	true	true		
	M	sens_out		false	false	false	false	false		
	C	stato		IDLE	IDLE	CHK_IN VER	INGR	IDLE		
	M	utente_rilevato					STD	STD		
	C	posti_std					1	1	0	
	C	posti_dis					1	1	1	
	C	posto_assegnato						POSTO_STD	POSTO_STD	
	M	transito_ok						true		

Figure 2: Simulatore – ingresso STD completo

Nello screenshot c'è il giro completo di un ingresso STD: metto **sens_in** a true e passo in **CHK_IN**, poi imposto l'utente e la macchina va in **VER**, da lì in **INGR** perché c'è posto, e infine con **transito_ok** torno in **IDLE** con **posti_std** sceso da 1 a 0. Una cosa che all'inizio confonde: l'animatore mostra una funzione solo a partire dallo stato in cui viene letta o scritta la prima volta, quindi nelle prime colonne certe righe sembrano mancare, ma è normale.

	Type	Functions	^	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	
	M	sens_in		false	true	true	true	true	false	false	false	false	false		
	M	sens_out		false	false	false	false	false	true	true	true	true	true		
	C	stato		IDLE	IDLE	CHK_IN VER	INGR	IDLE	CHK_OUT	TARIF	TARIF	USC	IDLE		
	M	utente_rilevato				STD	STD	STD	STD	STD	STD	STD	STD		
	C	posti_std				1	1	0	0	0	0	0	1		
	C	posti_dis				1	1	1	1	1	1	1	1		
	C	posto_assegnato					POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	NESSUNO	
	M	transito_ok					true	true	true	true	true	true	true		
	M	pagamento_ok								false	true	true			

Figure 3: Simulatore – uscita STD con pagamento

Qui invece c'è l'uscita, sempre nella stessa sessione. Si vede il passaggio per **TARIF**, che è la parte che riguarda solo gli STD, e alla fine **posti_std** torna a 1 e **posto_assegnato** torna a NESSUNO.

2.2 Scenari Avalla

Gli scenari sono questi:

Scenario	Cosa prova
test_abbonato.avalla	Giro completo di un ABBONATO: entra, e in uscita salta il pagamento.
test_disabile.avalla	Giro completo di un DISABILE su posto riservato; controllo che <code>posto_assegnato</code> resti <code>POSTO_DIS</code> per tutta la sosta e si azzeri solo in uscita.
test_pieno.avalla	Uno STD prende l'unico posto, un secondo STD viene respinto.
test_disabile_- fallback.avalla	Il fallback: due disabili di fila, il secondo trova i riservati finiti e prende un posto standard.
test_pagamento_- std.avalla	Uscita STD con un pagamento che va male (resta in <code>TARIF</code>) e poi uno che va bene.
test_pieno_- totale.avalla	Parcheggio pieno del tutto (uno STD più un disabile), il terzo utente viene respinto.
test_random_- 30steps.avalla	Generato in automatico: 30 step random fatti nel simulatore ed esportati con "export to avalla".

Il primo, `test_abbonato.avalla`, era già dato ma non funzionava. Aveva due errori: inizializzava `posti_std` a 5, mentre nel modello parto da 1, e controllava uno stato chiamato `PARK` che nel mio modello non esiste, visto che si chiama `IDLE`. Corretti quei due punti gira senza problemi.

Type	Functions	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13
C	step__	4	5	6	7	8	8	8	8	8	8
C	stato	IDLE	CHK_OUT	USC	IDLE	IDLE	IDLE	IDLE	IDLE	IDLE	IDLE
C	sens_in	false	false	false	false	false	false	false	false	false	false
C	result	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato	ATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO
C	posto_assegnato	_STD	POSTO_STD	POSTO_STD	POSTO_STD	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO
C	transito_ok	false	false	true	true	true	true	true	true	true	true
C	posti_std	0	0	0	1	1	1	1	1	1	1
C	sens_out	true	false	false	false	false	false	false	false	false	false

Figure 4: Animazione di `test_abbonato.avalla`

Nell'animazione dell'abbonato la cosa da guardare è la fase di uscita: da `CHK_OUT` si salta direttamente a `USC`, `TARIF` non compare proprio. È esattamente la differenza tra abbonato e utente `STD`.

Qui invece si vedono i due disabili uno dietro l'altro: il primo prende il posto riservato, il secondo lo trova occupato e finisce su quello standard. Tutti i check passano.

E qui il parcheggio saturo, con il terzo utente respinto e `parcheggioPieno` che passa a `true`.

2.3 AsmetaV – validazione con copertura (Vc)

Gli scenari girano con AsmetaV usando l'opzione che calcola anche la copertura, quella che la consegna chiama "Vc". La differenza rispetto alla validazione normale è che, oltre a dirti se i `check` passano, ti stampa anche quali regole del modello sono state davvero eseguite. Mettendo insieme i sei scenari scritti a mano si coprono tutte e 8 le regole di transizione, la main rule e tutti i rami di `r_gestione_VER`.

Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11
C	step__	0	1	2	3	4	5	6	7	8	9	9	9
C	stato		CHK_IN	VER	INGR	IDLE	CHK_IN	VER	INGR	IDLE	IDLE	IDLE	IDLE
C	sens_in		false	false	false	true	false	false	false	false	false	false	false
C	result		1	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato		DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE
C	posto_assegnato				POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD
C	transito_ok				true	false	false	false	true	true	true	true	true
C	posti_dis				0	0	0	0	0	0	0	0	0
C	posti_std									0	0	0	0

Figure 5: Animazione di test_disabile_fallback.avalla

Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13	State 14	State 15
C	step__	0	1	2	3	4	5	6	7	8	9	10	11	12	13	13	13
C	stato		CHK_IN	VER	INGR	IDLE	CHK_IN	VER	INGR	IDLE	CHK_IN	VER	NEG	IDLE	IDLE	IDLE	IDLE
C	sens_in		false	false	false	true	false	false	false	true	false	false	false	false	false	false	false
C	result		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato		STD	STD	STD	STD	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE
C	posto_assegnato				POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS
C	transito_ok				true	false	false	false	true	false	false	false	false	false	false	false	false
C	posti_std				0	0	0	0	0	0	0	0	0	0	0	0	0
C	posti_dis								0	0	0	0	0	0	0	0	0
C	auto_via												true	true	true	true	true

Figure 6: Animazione di test_pieno_totale.avalla

Il confronto con gli scenari generati a caso è la cosa più interessante di questa parte. Un primo export di 15 step random copriva solo 6 regole su 9: restavano fuori `r_gestione_NEG`, `r_gestione_TARIF` e `r_gestione_USC`. Ho raddoppiato a 30 step (`test_random_30steps.avalla`) e la rule coverage è arrivata al 100%, ma la branch coverage no: `r_gestione_USC` al 33%, `r_gestione_TARIF` al 50%, `r_gestione_IDLE` al 75%.

Il motivo si capisce guardando un caso concreto. Il ramo del pagamento fallito in `TARIF` vuole che il generatore metta `pagamento_ok` a false proprio mentre il sistema si trova in `TARIF`, e a caso capita raramente. Infatti quel ramo lo copre solo lo scenario scritto apposta, `test_pagamento_std`. La generazione random quindi aiuta, ma i casi limite bisogna ancora andarseli a cercare a mano.

2.4 Model checking (asmata/src/SmartParking_MC.asm)

Le CTLSPEC sono 10. Otto sono vere e stanno in tre gruppi: safety (cose che non devono mai succedere), raggiungibilità (cose che devono poter succedere) e liveness (cose che prima o poi devono succedere). Le altre due sono false apposta, per vedere il controesempio.

#	CTLSPEC	Categoria	Esito
1	<code>ag(posti_std >= 0)</code>	Safety	TRUE
2	<code>ag(posti_dis >= 0)</code>	Safety	TRUE

#	CTLSPEC	Categoria	Esito
3	ag((stato=INGR and posto_-assegnato=POSTO_STD) implies posti_-std>0)	Safety	TRUE
4	ag(posti_std <= 1)	Safety	TRUE
5	ag(posti_dis <= 1)	Safety	TRUE
6	ag((stato=INGR and posto_-assegnato=POSTO_DIS) implies posti_-dis>0)	Safety	TRUE
7	ef(stato = TARIF)	Raggiungibilità	TRUE
8	ag(stato=TARIF implies ef(stato=IDLE))	Liveness	TRUE
9	ag(stato != NEG)	Falsa (voluta)	FALSE
10	ag(stato=TARIF implies af(stato=IDLE))	Falsa (voluta)	FALSE

Table 5: Le 10 CTLSPEC sottoposte ad AsmetaSMV: 8 verificate, 2 volutamente false.

Le prime sei dicono in sostanza che i contatori non impazziscono: non vanno mai sotto zero, non vanno mai sopra la capacità, e non mando mai un'auto dentro su un tipo di posto che non ha più disponibilità. La settima dice che lo stato di pagamento è raggiungibile, l'ottava che da TARIF si riesce sempre a tornare in IDLE.

Sulle due false vale la pena spendere due parole in più.

La #9 dice "l'accesso non viene mai negato". NuSMV la smonta con un controesempio di 9 stati: entra un abbonato che si prende l'unico posto standard, arriva un secondo veicolo, la verifica non trova posto e va in NEG. La cosa interessante è che in quella traccia `posti_dis` è ancora 1: il controesempio, oltre a dire che NEG è raggiungibile, conferma che il posto disabile non viene dato a chi non ne ha diritto nemmeno quando è l'ultima risorsa rimasta.

La #10 è la stessa cosa della #8 ma con `af` al posto di `ef`. La #8 chiede che da TARIF *esista* un cammino che torna in IDLE, la #10 che ci si torni su *tutti* i cammini. Ed è falsa, perché niente vieta all'ambiente di tenere `pagamento_ok` a false per sempre: un utente che non paga mai. Il controesempio è proprio quello, il loop infinito in TARIF. Sono rimaste tutte e due nel modello perché messe una accanto all'altra fanno vedere bene la differenza tra i due operatori.

2.4.1 Model Advisor (opzionale)

Il Model Advisor passa anche lui per la traduzione in NuSMV, quindi sul modello principale non parte proprio: dà "Error Domain Integer not supported". Girando sul modello semplificato, quello che esce è questo:

- **MP1, MP3, MP4, MP7** non segnalano niente: nessun aggiornamento inconsistente, nessuna assegnazione banale, nessuna locazione da rimuovere o mai aggiornata.
- **MP2** ("conditional rule not complete") segnala invece tutte le regole di transizione. Sono però falsi positivi: l'advisor segnala ogni `if` senza `else`, ma in ASM l'`else` che manca vuol dire "non aggiornare niente", ed è esattamente quello che serve per i self-loop. `r_gestione_NEG` deve restare in NEG finché l'auto non se ne va, e `r_gestione_TARIF` deve restare in TARIF se il pagamento non passa. Con un `else` esplicito il comportamento sarebbe identico ma il codice più lungo.
- **MP5/MP6** dicono "Domain Posti could be reduced: element {2} could be removed". Anche questo non è un difetto: il 2 c'è apposta, come spiegato sopra, e l'advisor sta solo

```

** Coverage Info: **
** covered rules: **
SmartParking::r_gestione_USC()
-> branch coverage:      33.33%
-> rule coverage:        75.00%
-> update rule coverage: 75.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_CHK_IN()
-> branch coverage:      - (no branches to be covered)
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_INGR()
-> branch coverage:      33.33%
-> rule coverage:        71.43%
-> update rule coverage: 66.67%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_VER()
-> branch coverage:      50.00%
-> rule coverage:        60.00%
-> update rule coverage: 40.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_IDLE()
-> branch coverage:      75.00%
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_Main()
-> branch coverage:      87.50%
-> rule coverage:        88.24%
-> update rule coverage: - (no update rules to be covered)
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_CHK_OUT()
-> branch coverage:      50.00%
-> rule coverage:        66.67%
-> update rule coverage: 50.00%
-> forall rule coverage: - (no forall rules to be covered)
** NOT covered rules: **
SmartParking::r_gestione_NEG()
SmartParking::r_gestione_TARIF()

```

Figure 7: Report di copertura AsmetaV (Vc)

confermando che non viene mai raggiunto. È la stessa informazione delle CTLSPEC 4 e 5, per un'altra strada.

2.4.2 ATGT – generazione automatica di scenari (opzionale)

ATGT genera altri scenari in automatico, che stanno in `asmeta/src/abstracttests/`. Sono cinque file `testtest*.avalla` e si possono aprire direttamente dal repository.

Il discorso su quanto servano davvero è lo stesso già fatto per lo scenario random da 30 step, quindi non lo ripeto: si veda §2.3.

```

model checking (w execution) on /Users/francesco/Documents/INGEGNERIA/MAGISTRALE/TESTING/eclipse_asmeta_
Execution of NuSMV code ...
-----
> NuSMV -dynamic -coi -quiet SmartParking_MC.smv
-- specification AG posti_std >= 0 is true
-- specification AG posti_dis >= 0 is true
-- specification AG ((posto_assegnato = POSTO_STD & stato = INGR) -> posti_std > 0) is true
-- specification AG posti_std <= 1 is true
-- specification AG posti_dis <= 1 is true
-- specification AG ((stato = INGR & posto_assegnato = POSTO_DIS) -> posti_dis > 0) is true
-- specification EF stato = TARIF is true
-- specification AG (stato = TARIF -> EF stato = IDLE) is true
-- specification AG stato != NEG is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
  stato = IDLE
  transito_ok = false
  auto_via = false
  sens_out = false
  sens_in = false
  utente_rilevato = ABBONATO
  pagamento_ok = false
  posti_dis = 1
  posti_std = 1
  posto_assegnato = NESSUNO
-> State: 1.2 <-
  sens_in = true
-> State: 1.3 <-
  stato = CHK_IN
  sens_in = false
-> State: 1.4 <-
  stato = VER
-> State: 1.5 <-
  stato = INGR
  transito_ok = true
  posto_assegnato = POSTO_STD
-> State: 1.6 <-
  stato = IDLE
  transito_ok = false
  sens_in = true
  posti_std = 0
-> State: 1.7 <-
  stato = CHK_IN
  sens_in = false
-> State: 1.8 <-
  stato = VER
-> State: 1.9 <-
  stato = NEG
-- specification AG (stato = TARIF -> AF stato = IDLE) is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 2.1 <-
  stato = IDLE
  transito_ok = false
  auto_via = false
  sens_out = false
  sens_in = false
  utente_rilevato = ABBONATO
  pagamento_ok = false
  posti_dis = 1
  posti_std = 1
  posto_assegnato = NESSUNO
-> State: 2.2 <-
  sens_out = true
-> State: 2.3 <-
  stato = CHK_OUT
  sens_out = false
  utente_rilevato = STD
-- Loop starts here
-> State: 2.4 <-
  stato = TARIF
  utente_rilevato = ABBONATO
-> State: 2.5 <-
model checking (w execution) finished

```

Figure 8: Esito delle 10 CTLSPEC in AsmetaSMV

Type/Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13	State 14	State 15
C stato	IDLE	CHK_OUT	USC	USC	IDLE	CHK_OUT	USC	USC	IDLE	CHK_IN	VER	INGR	INGR	IDLE	CHK_IN	VER
M sens_in	false	false	false	false	false	false	false	false	true	true	true	true	true	true	true	
M sens_out	true	true	true	true	true	true	true	true	true	true	true	true	true	true	true	
M utente_rilevato		ABBONATO	ABBONATO	ABBONATO	ABBONATO	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	STD	STD	STD	STD	STD	
M transito_ok			false	true	true	true	false	true	true	true	true	false	true	true	true	
C posto_assegnato				NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD
C posti_std											1	1	1	0	0	0
C posti_dis											1	1	1	1	1	1

Figure 9: Esecuzione random-step di verifica manuale

```

MP2: Every case rule without otherwise must be complete
NONE

MP3: Choose rule is always/sometimes/never not empty
NONE

MP3: Forall rule is always/sometimes/never not empty
NONE

MP3: Conditional rule eval to true
NONE

MP4: No assignment is always trivial
NONE

MP5: For every domain element e, there exists a location which has value e
Domain Posti could be reduced in size. Elements {2} could be removed.

MP6: Every controlled location can take any value in its codomain
Function posti_dis does not take the values {2} of its domain. It could be defined over the smaller domain {0, 1}.
Function posti_std does not take the values {2} of its domain. It could be defined over the smaller domain {0, 1}.

MP7: a location could be removed
NONE

MP7: a controlled location is never updated
NONE

MP7: a controlled location could be static
NONE

model advising finished

```

Figure 10: Output del Model Advisor (estratto)

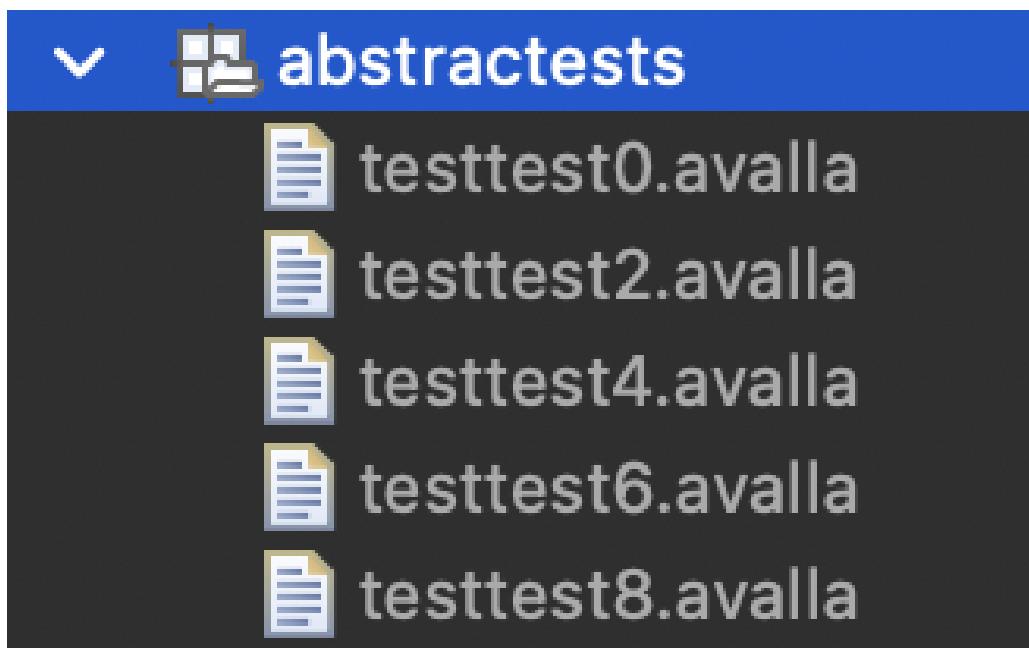


Figure 11: Elenco degli scenari generati da ATGT

3 Implementazione Java

3.1 Struttura del progetto Maven

Il progetto Java sta nella cartella `java/` ed è un progetto Maven normale, su Java 17. Le dipendenze principali sono Spring Boot con Vaadin 24 per la UI, JUnit 5 e Mockito per i test, JaCoCo per la copertura.

Il cuore sta nel package `it.tvsw.smartparking.core`, che contiene:

- `StatoSistema`, `TipoUtente`, `TipoPosto`: tre enum, uguali ai domini del modello ASM;
- `Parcheggio`: il nucleo, ed è l'unica classe con i contratti JML (Sezione 5);
- `SensorInput`: mette insieme i sei input monitorati di uno step;
- `Display`: un'interfaccia per le notifiche, che poi serve per Mockito;
- `SmartParkingFSM`: la macchina a stati vera e propria.

Fuori dal core ci sono solo la vista Vaadin e la classe di avvio di Spring Boot.

`Parcheggio` e `SmartParkingFSM` sono dichiarate `final` e tutte le classi del core sono serializzabili: non era così all'inizio, sono due modifiche arrivate dopo l'analisi statica e le spiego in Sezione 6.3.

3.2 La macchina a stati

`SmartParkingFSM` ha un metodo `step(SensorInput)` che fa la stessa cosa della main rule dell'ASM: guarda in che stato siamo e chiama il gestore giusto.

Listing 4: `step()` – dispatch sullo stato corrente

```

1 public void step(SensorInput input) {
2     switch (stato) {
3         case IDLE:      gestisciIdle(input); break;
4         case CHK_IN:    gestisciChkIn(); break;
5         case VER:       gestisciVer(input); break;
6         case NEG:       gestisciNeg(input); break;
7         case INGR:     gestisciIngr(input); break;
8         case CHK_OUT:   gestisciChkOut(input); break;
9         case TARIF:     gestisciTarif(input); break;
10        case USC:       gestisciUsc(input); break;
11    }
12 }
```

Ogni `case` corrisponde a una regola dell'ASM, quindi il confronto tra i due livelli si fa a colpo d'occhio. C'è però una differenza che è meglio dire subito. Nel modello ASM il contatore viene scalato solo al transito vero e proprio, cioè in `INGR`, mentre qui la classe `Parcheggio` decide e scala tutto insieme dentro `assegnaPosto`, che viene chiamato da `gestisciVer`. Negli scenari di test del progetto la differenza non si vede, perché nessuno va a controllare lo stato in mezzo ai due passi, ed è una scelta fatta apposta per tenere la classe più semplice. Ne riparlo meglio nell'ispezione del codice, in Sezione 6.

3.3 Interfaccia utente (Vaadin)

La UI è una vista sola, `MainView`, che mostra lo stato corrente e quanti posti sono ancora liberi. I sensori sono simulati con dei bottoni: “Arriva auto STD/DISABILE/ABBONATO”, “Transito”, “Pagamento”, “Auto va via” e “Auto in uscita”. Cliccandoli si fa avanzare la macchina a stati come se arrivassero gli input dai sensori veri.

Questa è la pagina appena aperta: stato `IDLE` e i due contatori a 1.

SmartParking - Simulatore

Stato: IDLE

Posti standard liberi: 1

Posti disabili liberi: 1

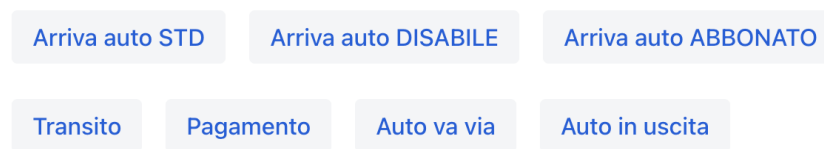


Figure 12: UI Vaadin – stato iniziale

SmartParking - Simulatore

Stato: IDLE

Posti standard liberi: 0

Posti disabili liberi: 1

Sistema pronto

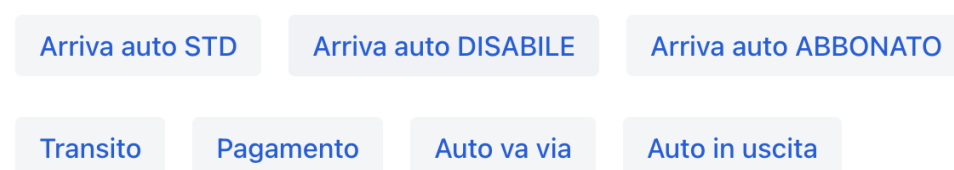


Figure 13: UI Vaadin – dopo un ingresso STD

Qui è stato cliccato “Arriva auto STD” e poi “Transito”. Siamo tornati in IDLE ma i posti standard sono scesi a 0.

Con il parcheggio pieno, se arriva un'altra auto compare il messaggio di accesso negato.

E questo è lo stato TARIF: l'utente STD sta uscendo e il sistema si è fermato ad aspettare il pagamento.

SmartParking - Simulatore

Stato: NEG

Posti standard liberi: 0

Posti disabili liberi: 1

Accesso negato

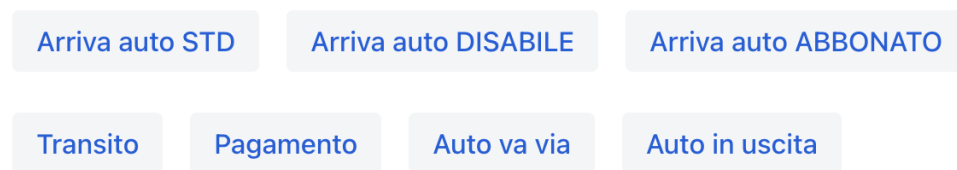


Figure 14: UI Vaadin – accesso negato a parcheggio pieno

SmartParking - Simulatore

Stato: TARIF

Posti standard liberi: 0

Posti disabili liberi: 1

In attesa di pagamento

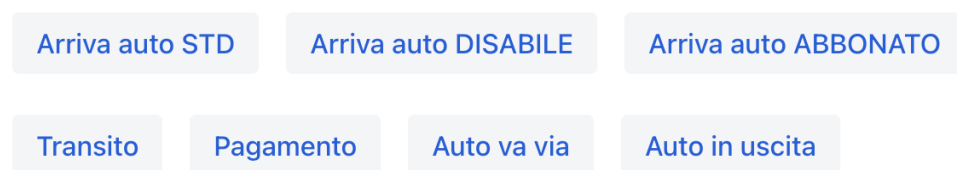


Figure 15: UI Vaadin – attesa pagamento

3.4 Test end-to-end con Selenium

La UI è provata anche con un test end-to-end in Selenium (`UISeleniumTest`). È taggato “ui” e sta fuori dalla build normale, perché per girare ha bisogno dell’applicazione già avviata e di Chrome installato, e in CI non ci sono.

Quello che fa è: apre la pagina con ChromeDriver in modalità headless, aspetta che Vaadin finisca di disegnare la pagina lato client (con `WebDriverWait`, altrimenti trova il DOM vuoto), clicca “Arriva auto STD” e controlla che lo stato mostrato diventi INGR oppure NEG. Per lanciarlo serve `mvn test -Dtest.groups=ui -Dtest.excludedGroups=`, oppure lo si fa partire da Eclipse come un normale test JUnit.

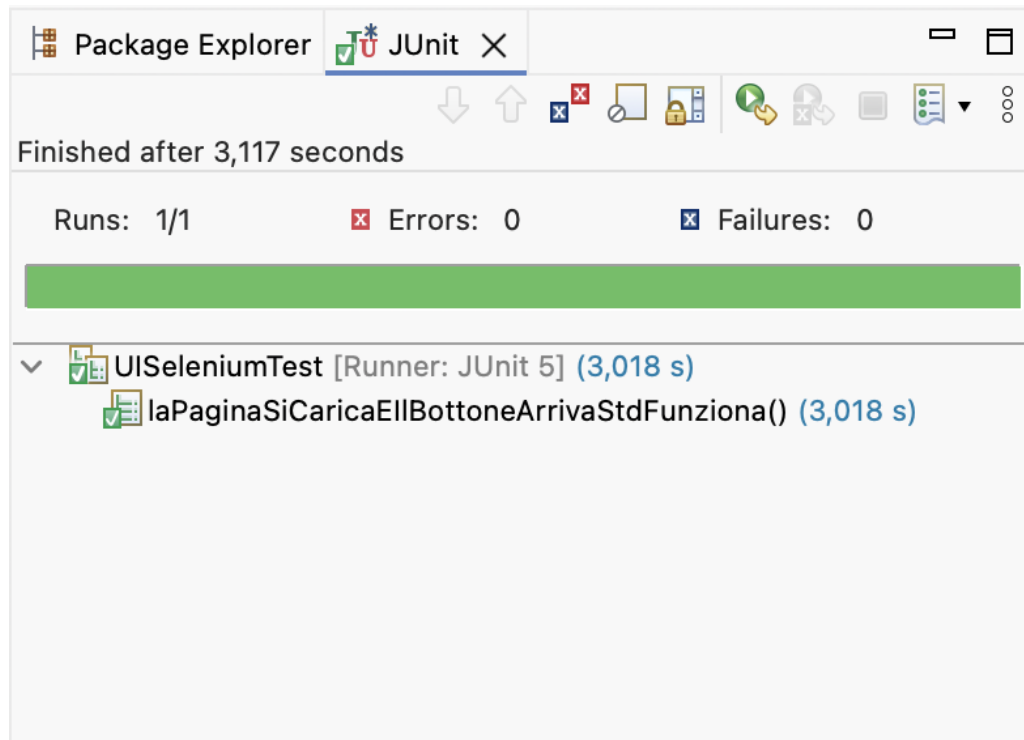


Figure 16: Test Selenium end-to-end superato

4 Testing Java

4.1 Struttura dei test

I test sono divisi in più classi, una per ogni cosa da coprire, invece che tutti insieme. Così se un test si rompe si capisce subito di che tipo di problema si tratta. Le classi sono queste:

Classe	Cosa copre
<code>ParcheggioTest</code>	Costruttori, preconditioni violate, tutti i rami di <code>assegnaPosto</code> e <code>liberaPosto</code> , più un test parametrico con CSV.
<code>SmartParkingFSMTest</code>	Tutti gli 8 stati e le transizioni, il fallback del disabile, il parcheggio pieno, i self-loop di NEG, INGR e TARIF.
<code>McdcVerificaTest</code>	Copertura MC/DC delle due decisioni composte di <code>assegnaPosto</code> .
<code>DisplayMockTest</code>	Le notifiche al <code>Display</code> , controllate con Mockito.
<code>ScenarioAvallaTest</code>	Traduzione di <code>test_pieno.avalla</code> in JUnit (model based testing).
<code>CombinatorialTest</code>	La tabella pairwise a 6 righe.

Classe	Cosa copre
UISeleniumTest	Smoke test della UI, taggato <code>ui</code> e fuori dalla build normale.

4.2 MC/DC

Dentro `assegnaPosto` ci sono due decisioni composte, cioè con più di una condizione legata da and/or, e su quelle serve la copertura MC/DC. La regola è che per ogni condizione serve una coppia di casi di test in cui cambia solo quella condizione e cambia anche il risultato della decisione. Vediamole una alla volta.

La prima è quella del ramo STD/ABBONATO: $D_1 = (\text{utente}=\text{STD} \vee \text{utente}=\text{ABBONATO}) \wedge \text{postiStd} > 0$. Le tre condizioni sono chiamate A, B e C nell'ordine in cui compaiono.

Caso	A	B	C	D_1	Esito	Coppia indipendente
TC1	T	F	T	T	POSTO_STD	base
TC2	T	F	F	F	NESSUNO	isola C (vs TC1)
TC3	F	F	T	F	POSTO_DIS	isola A (vs TC1)
TC4	F	T	T	T	POSTO_STD	isola B (vs TC3)

Table 7: MC/DC – Decisione 1.

Leggendola: TC1 è il caso base, un utente STD con un posto libero. TC2 cambia solo C (niente posti) e la decisione diventa falsa, quindi la coppia TC1–TC2 isola C. TC3 cambia solo A rispetto a TC1 ed è la coppia per A. TC4 confrontato con TC3 cambia solo B ed è la coppia per B. Con quattro casi sono coperte tutte e tre le condizioni.

La seconda decisione è quella del ramo DISABILE, cioè il fallback, con $C_1 = \text{postiDis} > 0$ e $C_2 = \text{postiStd} > 0$.

Caso	C_1	C_2	Esito	Coppia indipendente
TC5	T	—	POSTO_DIS	base
TC6	F	T	POSTO_STD	isola C_1 (vs TC5)
TC7	F	F	NESSUNO	isola C_2 (vs TC6)

Table 8: MC/DC – Decisione 2.

Qui in TC5 il valore di C_2 non conta, perché se c'è il posto disabile il secondo if non viene nemmeno valutato: per questo c'è un trattino.

4.3 Combinatorial testing (pairwise)

L'assegnazione del posto dipende da tre parametri: il tipo di utente (tre valori) e i due contatori, che nel modello valgono 0 oppure 1. Tutte le combinazioni sarebbero $3 \times 2 \times 2 = 12$, che sono poche e si potrebbero anche fare tutte, ma il punto dell'esercizio era usare il pairwise. Basta quindi coprire tutte le *coppie* di valori, e per farlo bastano 6 righe invece di 12.

#	tipoUtente	postiStd	postiDis	Esito atteso
1	STD	0	0	NESSUNO
2	STD	1	1	POSTO_STD
3	DISABILE	0	1	POSTO_DIS

#	tipoUtente	postiStd	postiDis	Esito atteso
4	DISABILE	1	0	POSTO_STD (fallback)
5	ABBONATO	0	1	NESSUNO
6	ABBONATO	1	0	POSTO_STD

Table 9: Tabella pairwise (copre tutte le coppie dei 3 parametri).

Le righe 3 e 5 sono le più interessanti, perché messe vicine fanno vedere la regola del posto riservato: stessa configurazione di contatori, ma il disabile entra e l'abbonato no.

4.4 Mockito

La FSM, ogni volta che cambia stato, manda un messaggio a un oggetto **Display**. Per controllare che questi messaggi partano davvero senza tirare in ballo la UI c'è Mockito: si crea un **Display** finto e poi con **verify** si controlla che il metodo **mostra** sia stato chiamato con il testo giusto.

Listing 5: Verifica delle notifiche con Mockito

```

1 @Test
2 void notificaAccessoNegatoQuandoParcheggioPieno() {
3     SmartParkingFSM fsm = new SmartParkingFSM(new Parcheggio(0, 0), display);
4     fsm.step(SensorInput.sensIn());
5     fsm.step(SensorInput.utente(TipoUtente.STD));
6     fsm.step(SensorInput.utente(TipoUtente.STD));
7     verify(display).mostra("Accesso negato");
8 }

```

Il test parte da un parcheggio già pieno, fa arrivare un'auto e controlla che al display arrivi davvero "Accesso negato".

4.5 Model Based Testing

Qui l'idea è prendere uno scenario già scritto sul modello astratto e riportarlo tale e quale sul codice vero. **test_pieno.avalla** è tradotto passo passo in **ScenarioAvallaTest**: ogni **set** dell'avalla diventa la preparazione dell'input, ogni **step** una chiamata a **step()** e ogni **check** un **assert**. Accanto a ciascuna riga c'è un commento con la riga originale dello scenario, così il confronto tra i due file si fa senza doverli tenere aperti tutti e due.

4.6 Esecuzione e copertura

Tutti i test del package **core** passano: sono 59, più quello Selenium che sta fuori dalla build normale.

La copertura si misura con JaCoCo lanciando **mvn verify** e guardando il report in **target/site/jacoco/index.html**. Dal conteggio sono escluse due cose: la UI Vaadin, provata end-to-end con Selenium e che JaCoCo non riesce a misurare, e la classe di avvio di Spring Boot, che sono tre righe di boilerplate. L'esclusione è scritta nel **pom.xml**.

Alla prima misurazione il package **core** stava al 98% di instruction e al 95% di branch. **Parcheggio** era già al 100% su tutte e due, il che tornava con il fatto che i suoi contratti erano già stati dimostrati con ESC. In **SmartParkingFSM** invece restavano fuori tre branch, e vale la pena guardarli uno per uno perché non sono tutti uguali:

- il self-loop di **gestisciUsc**, cioè il caso in cui in USC il transito non è ancora avvenuto;
- il ramo **display == null** di **cambiaStato**;

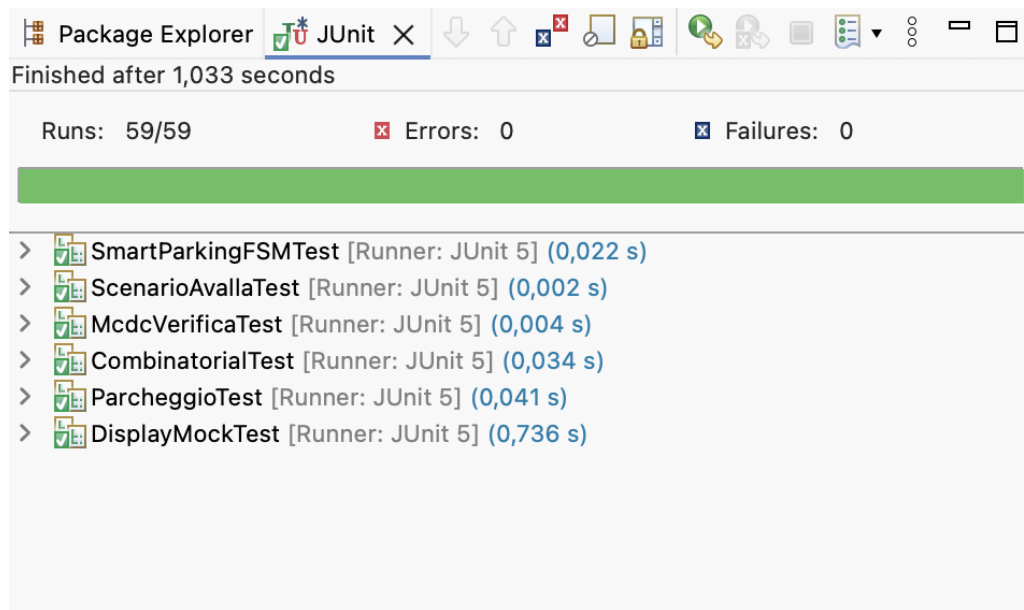


Figure 17: Esecuzione di tutta la suite JUnit

- il `default` dello switch di `step`.

I primi due erano buchi veri, casi che mi ero semplicemente dimenticato di provare, e li ho chiusi con due test in più (`uscRestaInUscFinoAlTransito` e `fsmFunzionaSenzaDisplay`). Il terzo invece non si può coprire: lo switch gestisce tutti e 8 i valori dell'enum, quindi in quel `default` non ci si arriva mai. L'ho lasciato lo stesso perché lancia un'eccezione ed è una rete di sicurezza se un giorno qualcuno aggiunge uno stato nuovo. È lo stesso tipo di ramo infattibile già incontrato con `AsmetaV` sul modello (§2.3).

Dopo i due test aggiunti, quel `default` è rimasto l'unico branch scoperto di tutto il package.

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cnty	Missed	Lines	Missed	Methods	Missed	Classes
it.tvsw.smartparking.core	8 of 602	98%	1 of 61	98%	1	73	1	147	0	39	0	6
Total	8 of 602	98%	1 of 61	98%	1	73	1	147	0	39	0	6

Figure 18: Report JaCoCo – totale (solo package core)

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cnty	Missed	Lines	Missed	Methods	Missed	Classes
SmartParkingFSM	96%	96%	1	35	1	72	0	16	0	1		
Parcheggio	100%	100%	0	21	0	37	0	6	0	1		
SensorInput	100%	n/a	0	14	0	21	0	14	0	1		
StatoSistema	100%	n/a	0	1	0	9	0	1	0	1		
TipoPosto	100%	n/a	0	1	0	4	0	1	0	1		
TipoUtente	100%	n/a	0	1	0	4	0	1	0	1		
Total	8 of 602	98%	1 of 61	98%	1	73	1	147	0	39	0	6

Figure 19: Report JaCoCo – dettaglio per classe del package core

4.7 Mutation testing con PIT

Arrivato al 98% di copertura mi restava un dubbio: la copertura dice che una riga è stata *eseguita* durante i test, non che qualcuno stia davvero *controllando* cosa fa. Un test che chiama un metodo e non verifica niente alza la copertura esattamente come un test scritto bene.

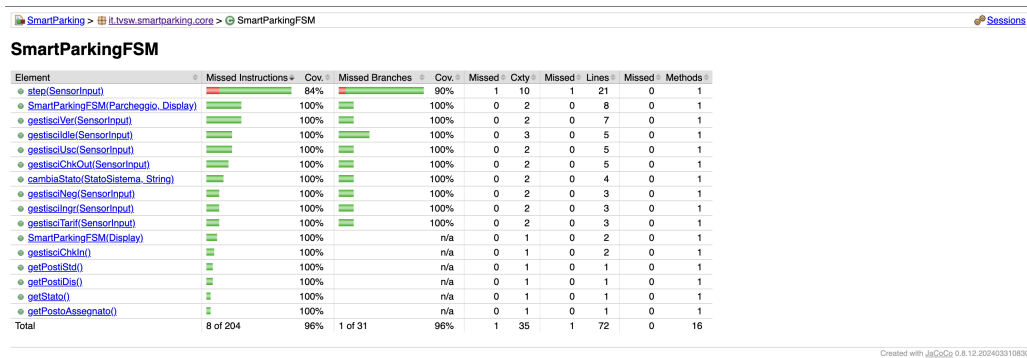


Figure 20: Report JaCoCo – dettaglio per metodo di SmartParkingFSM

Il mutation testing serve proprio a distinguere i due casi. PIT prende il bytecode e ci introduce piccoli difetti uno alla volta: cambia un `>` in `>=`, nega una condizione, sostituisce un valore di ritorno, cancella una chiamata a un metodo `void`. Per ogni difetto rilancia i test. Se i test falliscono il mutante è *ucciso*, e vuol dire che quella riga è sorvegliata da un assert. Se invece passano lo stesso il mutante *sopravvive*, e ho trovato un punto cieco.

Il plugin è configurato nel `pom.xml` sul solo package `core`, e si lancia così:

Listing 6: Esecuzione di PIT

```
$ mvn test-compile org.pitest:pitest-maven:mutationCoverage
```

Il risultato è che su **90 mutanti generati ne sono stati uccisi 90**, cioè un mutation score del 100%, con zero mutanti privi di copertura.

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
3	99% 127/128	100% 90/90	100% 90/90

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
it.tvsw.smartparking.core	3	99% 127/128	100% 90/90	100% 90/90

Report generated by [PIT](#) 1.15.8

Enhanced functionality available at [arcmutate.com](#)

Figure 21: Report PIT – riepilogo sul package core

Nel riepilogo si vedono due numeri diversi che è bene non confondere. La *line coverage* è 127 righe su 128, e quella riga scoperta è sempre il `default` irraggiungibile dello switch di cui sopra. La *mutation coverage* è 90 su 90, e dice tutt'altro: non quante righe sono state eseguite, ma su quante di esse un difetto introdotto di proposito è stato intercettato dai test.

Operatore di mutazione	Generati	Uccisi	%
NegateConditionals	26	26	100
VoidMethodCall	21	21	100
NullReturnVals	15	15	100
ConditionalsBoundary	9	9	100
BooleanTrueReturnVals	5	5	100
BooleanFalseReturnVals	5	5	100

Operatore di mutazione	Generati	Uccisi	%
Math	5	5	100
PrimitiveReturns	4	4	100
Totale	90	90	100

Table 10: Mutanti generati da PIT sul package `core`, per operatore.

Un mutante da vicino

Per capire cosa vuol dire in concreto “mutante ucciso” conviene guardarne uno solo. Prendiamo il ramo STD/ABBONATO di `assegnaPosto`, dove il codice originale è questo:

Listing 7: Codice originale

```

1 if (postiStd > 0) {
2     postiStd = postiStd - 1;
3     return TipoPosto.POSTO_STD;
4 }
5 return TipoPosto.NESSUNO;
```

L'operatore `ConditionalsBoundary` sposta il limite del confronto, trasformando `postiStd > 0` in `postiStd >= 0`. È una modifica di un carattere, e in tutte le esecuzioni con almeno un posto libero non cambia assolutamente niente: il codice mutato si comporta come l'originale.

La differenza salta fuori in un caso solo, quello con `postiStd == 0`. Lì l'originale salta il blocco e restituisce `NESSUNO`, mentre il mutante entra, decrementa il contatore portandolo a `-1` e restituisce `POSTO_STD`, cioè fa entrare un'auto in un parcheggio pieno.

Quel mutante viene ucciso dal primo caso della tabella pairwise (Tabella 9), l'unico che prova un utente STD con entrambi i contatori a zero. Ed è interessante notare che lo stesso difetto sarebbe stato intercettato anche dagli altri due livelli di verifica, ognuno a modo suo: il contatore a `-1` viola l'invariante `JML 0 <= postiStd` (Sezione 5) e viola la `CTLSPEC ag(posti_std >= 0)` verificata sul modello (§2.3). Tre tecniche diverse che convergono sullo stesso stato illegale.

Due operatori che valgono più degli altri

Due righe della tabella mi hanno fatto piacere più delle altre, perché ricollegano il risultato a lavoro fatto prima.

I 9 mutanti `ConditionalsBoundary` sono quelli che trasformano `postiStd > 0` in `postiStd >= 0` e simili. Ucciderli tutti vuol dire che i test distinguono davvero il caso “c'è un posto” dal caso “non ce n'è”, ed è esattamente quello che serviva costruire con la tabella MC/DC: senza quei casi di confine mirati, una mutazione sul limite sarebbe passata liscia.

I 21 mutanti `VoidMethodCall` sono le chiamate a `display.mostra(...)` cancellate. Sono metodi che non restituiscono niente, quindi cancellarli non cambia lo stato della FSM: un test che guarda solo lo stato finale non se ne accorgerebbe mai. Vengono uccisi grazie ai `verify` di Mockito, che è la conferma che quel pezzo di lavoro non era decorativo.

Quanto vale davvero questo 100%

Un punteggio pieno fa comodo ma va letto per quello che è, altrimenti si finisce per raccontarsi che il codice è perfetto.

Primo: il nucleo è piccolo e fatto apposta per essere verificabile. Non ci sono cicli, l'aritmetica è solo somma e sottrazione di 1, non ci sono strutture dati. Su un dominio del genere il numero di mutanti possibili è basso e coprirli tutti è realistico; su una classe con array e cicli sarebbe tutta un'altra storia.

Secondo, e più importante: PIT ha usato il suo insieme di operatori predefinito, quello che si vede nella Tabella 10. Attivando gruppi più aggressivi il numero di mutanti crescerebbe e qualcuno probabilmente sopravviverebbe. Quindi il 100% non significa “nessun difetto possibile sfugge ai miei test”, ma “nessuno dei difetti di questa famiglia sfugge ai miei test”. È una differenza che vale la pena tenere presente, perché è la stessa che c’è fra la copertura e il mutation testing, solo spostata di un livello.

Detto questo, la cosa che il risultato dimostra sul serio è quella che cercavo: quel 98% di copertura non è gonfiato. Le righe eseguite dai test sono anche righe i cui effetti vengono controllati.

5 JML e verifica statica (ESC)

Per la parte di design by contract è annotata con JML una classe sola, `Parcheggio`. È il nucleo del dominio: tiene i due contatori dei posti liberi e gestisce l’assegnazione e il rilascio, cioè fa quello che nel modello ASM fanno le regole `r_gestione_VER`, `r_gestione_INGR` e `r_gestione_USC`.

È piccola apposta e senza dipendenze da niente: dentro ci sono solo `int` ed `enum`. Il motivo è pratico, cioè che così OpenJML riesce a dimostrarla tutta, mentre su una classe che si tira dietro mezzo Spring non ci sarei mai arrivato. Tutto il resto del sistema (la FSM, la UI, il display, i sensori) è Java normale senza annotazioni, come chiedeva la consegna.

Una cosa scoperta sbattendoci contro: i campi `postiStd` e `postiDis` vanno dichiarati `private /*@ spec_public @*/`. Le clausole di specifica `public`, come le invarianti e le postcondizioni, possono nominare solo roba visibile a livello `public`, e se i campi restano semplicemente `private` OpenJML si ferma con un errore di visibilità. Con `spec_public` restano `private` per il codice ma diventano visibili alla specifica.

5.1 Invarianti

Le invarianti sono due e devono valere sempre, da dopo il costruttore in poi:

- i posti standard liberi stanno sempre tra 0 e `MAX_STD`;
- i posti disabili liberi stanno sempre tra 0 e `MAX_DIS`.

Listing 8: Invarianti di classe

```
//@ public invariant 0 <= postiStd && postiStd <= MAX_STD;
private /*@ spec_public @*/ int postiStd;

//@ public invariant 0 <= postiDis && postiDis <= MAX_DIS;
private /*@ spec_public @*/ int postiDis;
```

Sono le stesse identiche proprietà già verificate con il model checking, cioè `ag(posti >= 0)` e `ag(posti <= 1)`. Le ho scritte apposta uguali: così la stessa cosa finisce garantita a tre livelli diversi, nel modello con il model checker, nel codice con il contratto, e ancora nei test con JaCoCo.

5.2 Costruttori

I costruttori sono due. Quello di default garantisce lo stato iniziale del modello ASM, cioè tutti i posti liberi. Quello con i parametri serve nei test, per partire da una configurazione qualunque, e chiede che i valori passati siano già dentro i limiti dell’invariante.

Listing 9: Contratti dei costruttori

```
//@ ensures postiStd == MAX_STD && postiDis == MAX_DIS;
```

```
public Parcheggio() { ... }

/*@ requires 0 <= postiStdIniziali && postiStdIniziali <= MAX_STD;
   @ requires 0 <= postiDisIniziali && postiDisIniziali <= MAX_DIS;
   @ ensures postiStd == postiStdIniziali && postiDis == postiDisIniziali;
public Parcheggio(int postiStdIniziali, int postiDisIniziali) { ... }
```

Il `requires` del secondo costruttore dice cosa deve garantire chi chiama, ma non protegge a runtime: se qualcuno passa un valore fuori range senza aver fatto la verifica statica, il contratto non lo ferma. Per questo dentro c'è anche un controllo esplicito che lancia `IllegalArgumentException`, provato in `ParcheggioTest` con `assertThrows`.

5.3 Metodo assegnaPosto

Questo è quello con il contratto più lungo, perché deve dire tutto quello che fa `r_gestione_VER`. Le postcondizioni sono due, una per ramo.

Per STD e ABBONATO ci sono due esiti possibili: o c'era un posto standard libero, e allora torna `POSTO_STD` con il contatore sceso di 1, oppure non c'era e torna `NESSUNO` senza toccare niente.

Per il DISABILE gli esiti sono tre, perché in mezzo c'è il fallback: posto disabili se disponibile, altrimenti posto standard se disponibile, altrimenti `NESSUNO`.

Listing 10: Contratto di `assegnaPosto`

```
/*@ requires utente != null;
   @ ensures (utente == TipoUtente.STD || utente == TipoUtente.ABBONATO) ==>
   @      ((\old(postiStd) > 0 && \result == TipoPosto.POSTO_STD
   @      && postiStd == \old(postiStd) - 1 && postiDis == \old(postiDis))
   @      || (\old(postiStd) == 0 && \result == TipoPosto.NESSUNO
   @      && postiStd == \old(postiStd) && postiDis == \old(postiDis)));
   @ ensures utente == TipoUtente.DISABILE ==>
   @      ((\old(postiDis) > 0 && \result == TipoPosto.POSTO_DIS
   @      && postiDis == \old(postiDis) - 1 && postiStd == \old(postiStd))
   @      || (\old(postiDis) == 0 && \old(postiStd) > 0
   @      && \result == TipoPosto.POSTO_STD
   @      && postiStd == \old(postiStd) - 1 && postiDis == \old(postiDis))
   @      || (\old(postiDis) == 0 && \old(postiStd) == 0
   @      && \result == TipoPosto.NESSUNO
   @      && postiStd == \old(postiStd) && postiDis == \old(postiDis)));
public TipoPosto assegnaPosto(TipoUtente utente) { ... }
```

Il `\old` serve per parlare del valore che il contatore aveva prima della chiamata, altrimenti non si potrebbe dire “è sceso di uno”.

Una cosa che all'inizio mancava: in ogni esito è scritto anche che l'*altro* contatore resta com'era. Senza quel pezzo il contratto sarebbe sottospecificato, cioè un'implementazione che assegna il posto giusto ma nel frattempo tocca anche l'altro contatore lo rispetterebbe lo stesso.

5.4 Metodo liberaPosto

Questo corrisponde al rilascio del posto in `r_gestione_USC`. Chiede che il posto non sia null e che il contatore corrispondente non sia già al massimo, e garantisce che venga incrementato di 1 solo quello giusto. Con `NESSUNO` non fa niente e lo stato resta identico.

Listing 11: Contratto di `liberaPosto`

```
/*@ requires posto != null;
   @ requires posto == TipoPosto.POSTO_STD ==> postiStd < MAX_STD;
   @ requires posto == TipoPosto.POSTO_DIS ==> postiDis < MAX_DIS;
   @ ensures posto == TipoPosto.POSTO_STD ==>
   @      postiStd == \old(postiStd) + 1 && postiDis == \old(postiDis);
```

```
//@ ensures posto == TipoPosto.POSTO_DIS ==>
//@     postiDis == \old(postiDis) + 1 && postiStd == \old(postiStd);
//@ ensures posto == TipoPosto.NESSUNO ==>
//@     postiStd == \old(postiStd) && postiDis == \old(postiDis);
public void liberaPosto(TipoPosto posto) { ... }
```

Quel secondo e terzo **requires** non c'erano nella prima versione e sono il motivo per cui la prova non chiudeva: ne parlo tra poco.

5.5 Metodi getter

I due getter sono annotati `/*@ pure @*/`, che vuol dire senza effetti collaterali. Serve perché un metodo non puro dentro una clausola JML non si può usare. La postcondizione dice solo che restituiscono il campo, niente di più.

Listing 12: Getter puri

```
//@ ensures \result == postiStd;
public /*@ pure @*/ int getPostiStd() { ... }

//@ ensures \result == postiDis;
public /*@ pure @*/ int getPostiDis() { ... }
```

5.6 Verifica dei contratti con OpenJML (ESC)

La verifica dei contratti è stata fatta con OpenJML (release 21.0.27, build nativa per macOS arm64) in modalità `-esc`. Quello che fa il tool è tradurre codice e contratti in formule logiche e passarle al solver Z3, che deve dimostrare che *qualunque* esecuzione rispetta postcondizioni e invarianti. È una cosa diversa dai test: i test provano alcuni casi, qui invece o si dimostra per tutti o non si dimostra.

Il comando, dalla root del progetto, è questo:

Listing 13: Comando di verifica ESC

```
$ openjml --esc --progress \
  -cp java/src/main/java \
  java/src/main/java/it/tvsw/smartparking/core/Parcheggio.java
```

Un contratto che all'inizio non chiudeva

Arrivare a “tutti Verified” non è stato immediato, e il caso di `liberaPosto` secondo me è quello che spiega meglio come funziona la cosa.

La prima versione era questa, con la postcondizione che descrive l'incremento ma senza il **requires** sulla capacità:

Listing 14: Prima versione di liberaPosto (senza il requires di capacità)

```
//@ requires posto != null;
//@ ensures posto == TipoPosto.POSTO_STD ==>
//@     postiStd == \old(postiStd) + 1 && postiDis == \old(postiDis);
public void liberaPosto(TipoPosto posto) { ... }
```

Il solver, non avendo nessun vincolo sul valore di partenza, prova anche il caso `postiStd == MAX_STD`. Lì `postiStd + 1` fa `MAX_STD + 1`, che viola l'invariante di classe. Quindi non riesce a ristabilire l'invariante all'uscita del metodo e risponde così:

Listing 15: Errore ESC sulla prima versione

```
Parcheggio.java:.. warning: The prover cannot establish an assertion
(InvariantExit) in method liberaPosto
```

```
postiStd == \old(postiStd) + 1 ...
Associated declaration: invariant 0 <= postiStd && postiStd <= MAX_STD
```

La soluzione è rafforzare la preconditione, in modo che sia chi chiama a garantire che c'è spazio per incrementare:

Listing 16: Versione corretta: il requires di capacità chiude la prova

```
//@ requires posto == TipoPosto.POSTO_STD ==> postiStd < MAX_STD;
//@ requires posto == TipoPosto.POSTO_DIS ==> postiDis < MAX_DIS;
```

Con `postiStd < MAX_STD` assunto all'ingresso, dopo l'incremento vale `postiStd <= MAX_STD`, l'invariante regge e il metodo passa a Verified.

Il senso della cosa, secondo me, è questo: ogni volta che c'è un'operazione aritmetica bisogna aver vincolato prima tutto quello che ci entra, con un `requires` o con un'invariante. `Parcheggio` chiude tutto perché ha due soli interi e tutti e due sono limitati; su un nucleo con array o contatori liberi la stessa cosa diventa molto più difficile. L'altro ostacolo era di tutt'altro tipo, cioè la visibilità dei campi private, risolto con `spec_public` come detto all'inizio.

```
/Users/francesco/Documents/INGEGNERIA/MAGISTRALE/TESTING/openjml-macos-arm64-21.0.27/openjml \
--esc --progress \
-cp java/src/main/java \
java/src/main/java/it/tvsw/smartparking/core/Parcheggio.java
Proving methods in it.tvsw.smartparking.core.Parcheggio
Starting proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio() with prover z3-4.3.X
Method assertions are validated [1,14 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio() with prover z3-4.3.X - no warnings [1,26 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio(int,int) with prover z3-4.3.X
Method assertions are validated [1,21 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio(int,int) with prover z3-4.3.X - no warnings [1,30 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.assegnaPosto(it.tvsw.smartparking.core.TipoUtente) with prover z3-4.3.X
Method assertions are validated [1,23 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.assegnaPosto(it.tvsw.smartparking.core.TipoUtente) with prover z3-4.3.X - no warnings [1,30 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.getPostiDis() with prover z3-4.3.X
Method assertions are validated [1,03 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.getPostiDis() with prover z3-4.3.X - no warnings [1,04 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.getPostiStd() with prover z3-4.3.X
Method assertions are validated [1,00 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.getPostiStd() with prover z3-4.3.X - no warnings [1,05 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.liberaPosto(it.tvsw.smartparking.core.TipoPosto) with prover z3-4.3.X
Method assertions are validated [1,19 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.liberaPosto(it.tvsw.smartparking.core.TipoPosto) with prover z3-4.3.X - no warnings [1,22 secs]
Completed proving methods in it.tvsw.smartparking.core.Parcheggio [7,18 secs]
Proving methods in it.tvsw.smartparking.core.TipoUtente
Starting proof of it.tvsw.smartparking.core.TipoUtente.TipoUtente() with prover z3-4.3.X
Method assertions are validated [1,00 secs]
Completed proof of it.tvsw.smartparking.core.TipoUtente.TipoUtente() with prover z3-4.3.X - no warnings [1,07 secs]
Completed proving methods in it.tvsw.smartparking.core.TipoUtente [1,07 secs]
Proving methods in it.tvsw.smartparking.core.TipoPosto
Starting proof of it.tvsw.smartparking.core.TipoPosto.TipoPosto() with prover z3-4.3.X
Method assertions are validated [1,06 secs]
Completed proof of it.tvsw.smartparking.core.TipoPosto.TipoPosto() with prover z3-4.3.X - no warnings [1,07 secs]
Completed proving methods in it.tvsw.smartparking.core.TipoPosto [1,07 secs]
Summary:
Valid:      8
Invalid:    0
Infeasible: 0
Timeout:    0
Error:      0
Skipped:    0
TOTAL METHODS: 8
Classes:    3 proved of 3
DURATION:   9,6 secs
(base) francesco@iPhone Progetto_Testing %
```

Figure 22: Output di openjml -esc: tutti i metodi Verified

Alla fine tutti e sei i metodi (i due costruttori, `assegnaPosto`, `liberaPosto` e i due getter) risultano Verified.

5.7 Verifica dinamica: i contratti a runtime (RAC)

Vedere scritto “Verified” per tutti i metodi è comodo, ma da solo non mi diceva se i contratti stessero davvero controllando qualcosa o se fossero solo scritti bene. È lo stesso dubbio che mi era venuto con la pipeline verde della CI: passa, ma se rompo qualcosa se ne accorge?

Per togliermelo ho usato OpenJML nell'altra modalità, `-rac` (Runtime Assertion Checking). Qui il tool non dimostra niente in anticipo: infila i controlli dentro il bytecode, e i contratti vengono verificati mentre il programma gira. Se una clausola salta, l'esecuzione si ferma dicendo quale.

Il programma di prova è `DemoRac` (package `it.tvsw.smartparking.jml`), che serve solo a questo e non viene chiamato né dalla FSM né dalla UI. Fa tre cose in fila: prima un uso legittimo, che non deve dare nessun errore, e poi due violazioni fatte apposta.

Listing 17: Le due violazioni volontarie in DemoRac

```

1 // 1. Viola il requires del costruttore: MAX_STD vale 1, qui ne passo 5
2 Parcheggio p = new Parcheggio(5, 0);
3
4 // 2. Viola il requires di liberaPosto: il contatore e' gia' al massimo,
5 //   perche' non e' entrato nessuno, quindi l'incremento sfonderebbe l'
6 //   invariante
7 Parcheggio q = new Parcheggio();
8 q.liberaPosto(TipoPosto.POSTO_STD);

```

I comandi per compilare ed eseguire sono questi:

Listing 18: Compilazione ed esecuzione con RAC

```

$ openjml --rac -d out \
  java/src/main/java/it/tvsw/smartparking/core/*.java \
  java/src/main/java/it/tvsw/smartparking/jml/DemoRac.java

$ java -cp out:$OPENJML_HOME/jmlruntime.jar \
  it.tvsw.smartparking.jml.DemoRac

```

La seconda violazione è quella che mi interessava di più, perché è esattamente il caso che aveva fatto fallire la prova ESC prima che aggiungessi il **requires** sulla capacità. Prima quel caso era un buco nella specifica e il solver me lo segnalava come “cannot establish InvariantExit”; adesso che il **requires** c’è, lo stesso caso diventa una preconditione violata dal chiamante, e a runtime si vede subito. Le due modalità guardano lo stesso identico punto da due lati: ESC dice “non riesco a dimostrare che vale sempre”, RAC dice “ecco l’esecuzione in cui non vale”.

6 Ispezione del codice e analisi statica

Questa parte l’ho affrontata in tre passaggi diversi, perché ognuno trova cose che gli altri due si perdono: prima l’ispezione manuale con la checklist vista a lezione, poi una seconda lettura fatta fare a un LLM, e infine l’analisi automatica con SpotBugs.

6.1 Ispezione con la checklist

La prima passata l’ho fatta con la checklist di code inspection del corso, andando in ordine sulle nove categorie. Sotto riporto per ognuna cosa è venuto fuori. Alcune voci sul mio codice non si applicano proprio, e lo dico invece di spuntarle a vuoto.

1. Specification / Design. La funzionalità descritta nei requisiti c’è tutta: ogni requisito da RF1 a RF14 ha il suo corrispettivo in **SmartParkingFSM** o in **Parcheggio**. Di funzionalità in più rispetto alla specifica non ne ho trovate: i due metodi che sembravano candidati, **SensorInput.nessunEvento()** e il costruttore **SmartParkingFSM(Display)**, sono entrambi usati dai test, il primo per provare i self-loop (uno step senza nessun sensore attivo) e il secondo per costruire la FSM con il parcheggio di default.

2. Initialization and Declarations. Tutti i campi sono inizializzati nel costruttore, e quelli che non cambiano mai (**parcheggio**, **display**, tutti i campi di **SensorInput**) sono **final**. I tipi sono quelli giusti: gli stati e i tipi di utente sono enum e non interi o stringhe, quindi un valore fuori dominio non è nemmeno rappresentabile. Tutti i campi sono **private**, e **SensorInput** è dichiarata **final** e immutabile.

3. Method Calls. Qui è uscita la segnalazione più concreta di tutta la checklist, e non l’aveva vista nessun altro controllo. Ci sono due firme con **parametri adiacenti dello stesso tipo**:

Listing 19: Firme con parametri adiacenti dello stesso tipo

```
1 public Parcheggio(int postiStdIniziali, int postiDisIniziali)
2
3 public SensorInput(boolean sensIn, boolean sensOut, boolean transitoOk,
4                   boolean autoVia, TipoUtente utenteRilevato, boolean
                      pagamentoOk)
```

Se in una chiamata scambio i due `int`, o due dei quattro `boolean`, il codice compila senza un fiato e il comportamento cambia. È esattamente il tipo di errore che la categoria 3 della checklist va a cercare, e che né il compilatore né i contratti JML possono intercettare, perché a livello di tipi la chiamata è corretta.

Su `SensorInput` il problema in pratica non si presenta, perché il costruttore non lo chiama mai nessuno direttamente: si passa sempre dai metodi statici (`sensIn()`, `transito()`, `pagamento(esito)...`), che hanno zero o un parametro e un nome che dice cosa fanno. È una protezione che avevo messo per comodità di scrittura dei test, e che a posteriori risolve anche questo.

Su `Parcheggio(int, int)` invece il rischio resta. L’ho lasciato così perché quel costruttore lo usano solo i test, dove i valori sono scritti in chiaro riga per riga e un eventuale scambio farebbe fallire subito l’assert. In un sistema vero la soluzione sarebbe un tipo dedicato per i due contatori, o un costruttore con *builder*.

4. Arrays. Non applicabile: nel progetto non c’è nessun array né nessuna collezione. Lo stato è tenuto in due interi e in due enum. Di conseguenza non ci sono errori di indice off-by-one né rischi di sfiorare i limiti.

5. Object Comparison. Tutti i confronti tra enum sono fatti con `==`, che per gli enum Java è corretto e anzi preferibile a `equals`. Stringhe da confrontare non ce ne sono: le uniche che compaiono sono i messaggi passati al display, che vengono solo scritti, mai testati per uguaglianza nel codice di produzione.

6. Output Format. I messaggi del display sono coerenti e senza errori di battitura. Due osservazioni minori. La prima è che alcune stringhe sono ripetute in più punti (“Sistema pronto” compare tre volte, una per ogni ritorno in IDLE): andrebbero in costanti, ma su quattro messaggi in croce ho lasciato stare. La seconda è che “Sbarra aperta” e “Sbarra di uscita aperta” sembrano un’incoerenza e invece sono volute, perché distinguono le due sbarre.

7. Computation, Comparisons and Assignments. Divisioni non ce ne sono, quindi il problema del denominatore a zero non si pone. L’aritmetica è solo somma e sottrazione di 1 sui due contatori, e non può andare in overflow perché i contatori sono limitati tra 0 e la capacità massima, cosa garantita in tre modi diversi: dall’invariante JML, dai controlli a runtime in `liberaPosto` e dalle CTLSPEC sul modello. Vale la pena dirlo perché l’overflow è il classico punto in cui la verifica ESC si inceppa, e qui non succede proprio grazie a quei vincoli.

8. Exceptions. Nel progetto non c’è nemmeno un `try/catch`. È una scelta: le uniche eccezioni lanciate (`IllegalArgumentException` e `IllegalStateException`) segnalano errori di programmazione, non condizioni che possono capitare a runtime, quindi devono propagare e non essere nascoste. Un punto di attenzione però c’è: nei gestori dei bottoni della UI Vaadin non c’è nessuna gestione, quindi un’eccezione lì diventerebbe una pagina di errore del framework. Per un prototipo va bene, in una versione vera ci vorrebbe un messaggio all’utente.

9. Flow of Control. Lo `switch` di `step` ha tutti i `case` chiusi da `break` e un ramo `default` che lancia eccezione, quindi niente fall-through accidentale. Il problema del *dangling else* non si presenta: l’unico punto con due condizioni in fila, `gestisciIdle`, usa un `if/else` if esplicito.

Infine, nel package `core` non c'è **nessun ciclo**: la FSM avanza di un passo per chiamata e l'iterazione, se serve, la fa il chiamante. Questo spiega anche perché nella parte JML (Sezione 5) non compare nessuna `loop_invariant`: non essendoci loop, non c'è niente da invariantizzare.

6.2 Seconda lettura con un LLM

La consegna chiedeva di fare l'ispezione “con chatgpt o LLM in generale”, quindi dopo la checklist ho usato ChatGPT come se fosse un collega a cui far dare una seconda occhiata al codice. Gli ho passato tutto il sorgente dei package `core` e `ui` e gli ho chiesto di leggerlo cercando nomi poco chiari, casi limite non gestiti, possibili `NullPointerException` e punti in cui il codice si allontana dal modello ASM.

Le cose che sono uscite non le ho prese per buone così com'erano: le ho valutate una per una, decidendo di volta in volta se correggerle, mitigarle o lasciarle stare spiegando perché. L'LLM ha segnalato i punti su cui ragionare, le decisioni le ho prese io.

1. **Manca il controllo di null su `utenteRilevato` in `CHK_OUT`**: il confronto restituisce false senza dire niente, e il sistema va in USC come se l'utente non fosse STD. L'ho lasciata così per restare fedele all'ASM, dove il dominio `TipoUtente` non prevede l'assenza del dato; in più tutti i punti che chiamano quel metodo impostano sempre l'utente.
2. **`TipoPosto.NESSUNO` vuol dire due cose diverse**: in `assegnaPosto` significa “non c'è posto”, in `liberaPosto` significa “non fare niente”. Chiarito nel Javadoc, invece di inventare un tipo separato che avrebbe appesantito una classe che doveva restare semplice.
3. **`liberaPosto` lancia eccezione se il contatore è già al massimo**. Questa l'ho tenuta apposta: preferisco un crash che si vede subito a un contatore che va oltre la capacità senza dire niente, violando l'invariante. Il caso è comunque coperto da un test.
4. **Il decremento avviene in un momento diverso rispetto all'ASM**: qui `assegnaPosto` decide e scala insieme, nell'ASM si scala al transito. Negli scenari di test la differenza non si vede, quindi resta la versione più semplice, scritta nel Javadoc della classe.
5. **Non c'è thread-safety**. Va bene così, perché c'è un varco solo pilotato da una FSM sequenziale, esattamente come nel modello. Resta segnata come limite se un giorno si volessero gestire più varchi.
6. **`posto_assegnato` è una memoria da un posto solo, e il difetto c'è anche nel modello ASM**. Questa è l'osservazione più importante di tutte. Sia la FSM Java sia l'ASM si ricordano un solo posto assegnato, ma nel parcheggio ci stanno due veicoli, uno standard e uno disabile. Se entra uno STD, poi entra un DISABILE che sovrascrive lo slot, e poi esce il primo, viene rilasciato il posto sbagliato e i contatori si sballano.

Il punto è che l'implementazione è fedele alla specifica: è la specifica stessa che dà per scontato di tracciare un veicolo alla volta, senza mai associare un veicolo al suo posto. Per sistemarlo servirebbe una mappa `veicolo→posto` sia nell'ASM sia in Java. L'ho lasciato e documentato come limite noto, perché mi sembra l'esempio più utile di tutto il progetto: è un difetto che il testing di conformità non può trovare, visto che il codice fa esattamente quello che dice il modello, e che salta fuori solo leggendo il codice con occhio critico.

7. **I bottoni della UI sono sempre attivi**: `MainView` non li disabilita in base allo stato, quindi si può cliccare “Pagamento” mentre siamo in IDLE. Non succede niente di male perché quegli step finiscono assorbiti dai self-loop della FSM, cioè proprio dagli `if` senza `else` che il Model Advisor segnalava come MP2 (§2.3). Per un prototipo va bene; in una versione vera si metterebbe un `setEnabled` legato allo stato.

6.3 Analisi automatica con SpotBugs

L'ultimo passaggio è il tool automatico. Ho usato SpotBugs (il successore di FindBugs, che ormai è fermo), configurato nel `pom.xml` con `effort=Max` e `threshold=Low`, cioè il massimo della sensibilità, così da vedere anche le segnalazioni di priorità bassa. Si lancia con `mvn spotbugs:check`.

Prima di arrivare al risultato ho perso un po' di tempo con un problema che non c'entra niente col codice: sulla mia macchina Maven girava di default con una JDK 26, e SpotBugs non riusciva a leggerne il bytecode (`Unsupported class file major version 70`). Il risultato era ingannevole, perché la build falliva ma con zero classi analizzate e zero bug: sembrava un errore mio. Lanciandolo con la JDK 17, la stessa che usa la CI, è andato a posto.

Prima passata: 7 bug

Tipo	Priorità	Dove
CT_CONSTRUCTOR_THROW	Medium	I due costruttori di <code>Parcheggio</code>
CT_CONSTRUCTOR_THROW	Medium	I due costruttori di <code>SmartParkingFSM</code>
SE_BAD_FIELD	Medium	<code>MainView</code> , campo <code>fsm</code>
SE_NO_SERIALVERSIONID	Low	<code>MainView</code>
SE_NO_SERIALVERSIONID	Low	<code>Application</code>

Table 11: Le 7 segnalazioni di SpotBugs (le prime due righe valgono 2 bug ciascuna).

Si dividono in due gruppi con storie molto diverse.

I quattro CT_CONSTRUCTOR_THROW. Riguardano tutti e quattro i costruttori che validano i parametri e lanciano `IllegalArgumentException`. SpotBugs avverte che un costruttore che lancia lascia l'oggetto parzialmente costruito, e questo apre la porta a un *finalizer attack*: una sottoclasse malevola sovrascrive `finalize()` e si ritrova in mano il riferimento a un oggetto che non ha mai superato i controlli.

Qui c'è un cortocircuito che mi ha fatto pensare. Quei `throw` sono esattamente i controlli difensivi che ho messo per far rispettare a runtime i `requires JML` (Sezione 5): cioè la cosa che il tool segnala è la stessa che rende il codice robusto. Toglierli sarebbe stato un peggioramento.

La soluzione giusta non è rinunciare alla validazione ma togliere il presupposto dell'attacco, cioè l'ereditarietà: ho dichiarato `final` sia `Parcheggio` sia `SmartParkingFSM`. Se la classe non si può estendere, nessuno può sovrascrivere `finalize()` e l'attacco non è più possibile. Prima di farlo ho controllato che nessuno le estendesse e che Mockito, in `DisplayMockTest`, mocchi soltanto l'interfaccia `Display`, che invece resta com'era.

I tre bug di serializzazione. Questo gruppo è quello che vale davvero, perché è l'unico punto in cui l'analisi automatica ha trovato un difetto che né la checklist né l'LLM avevano visto.

`SE_BAD_FIELD` segnala che `MainView` tiene un campo `fsm` non serializzabile. Sembra una pedanteria, e invece no: Vaadin mette l'intero albero dei componenti nella sessione HTTP, quindi con la replicazione di sessione o al riavvio del server quella vista non si riesce a serializzare e l'applicazione si pianta. È un difetto vero, solo latente, perché in locale con un'istanza sola non si manifesta mai.

Il motivo per cui gli altri due controlli se l'erano perso è chiaro a posteriori: per accorgersene bisogna sapere come Vaadin gestisce la sessione, e né la checklist né la lettura del codice guardavano lì. Il tool invece conosce i contratti delle librerie.

Ho reso `Parcheggio`, `SmartParkingFSM` e `SensorInput` serializzabili. Ho dovuto far estendere `Serializable` anche all'interfaccia `Display`, perché in `MainView` viene passata come lambda

(`messagingLabel::setText`) e una lambda è serializzabile solo se lo è la sua interfaccia funzionale. Infine ho aggiunto `serialVersionUID` a tutte le classi coinvolte, `MainView` e `Application` comprese.

Seconda passata: 0 bug

Dopo le correzioni `SpotBugs` riporta `BugInstance size is 0`, i 59 test del core restano tutti verdi e la copertura JaCoCo non si muove (98% instruction e 98% branch, sempre con quell'unico `default` irraggiungibile come solo branch scoperto).

6.4 Bilancio

Fra i tre passaggi sono uscite quindici osservazioni: sette dalla checklist e dall'LLM, sette da `SpotBugs`, più il difetto della specifica. Ed è interessante che ognuno dei tre metodi abbia trovato cose che gli altri due non potevano vedere.

La **checklist** è meccanica e va per categorie, quindi mi ha costretto a guardare le *firme* dei metodi, che leggendo il codice per capire cosa fa non guardi mai: da lì è saltato fuori il problema dei parametri adiacenti dello stesso tipo.

L'**LLM** ragiona sul significato, e ha trovato il difetto della memoria a slot singolo. Una checklist non avrebbe potuto vederlo, perché non viola nessuna regola: è il modello a essere incompleto.

SpotBugs conosce i contratti delle librerie, e ha trovato il problema di serializzazione di `Vaadin`, che gli altri due non avevano gli strumenti per notare.

Fra tutte, quella che conta di più resta il difetto della memoria a slot singolo, perché né il testing di conformità né la verifica ESC riuscivano a vederlo. I contratti JML di `Parcheggio` restano rispettati anche nell'esecuzione che rilascia il posto sbagliato, dato che il problema sta più in alto, a livello di FSM e di modello. Il model checking e l'ESC dimostrano quello che è stato formalizzato; l'ispezione trova quello che nella formalizzazione non c'è.

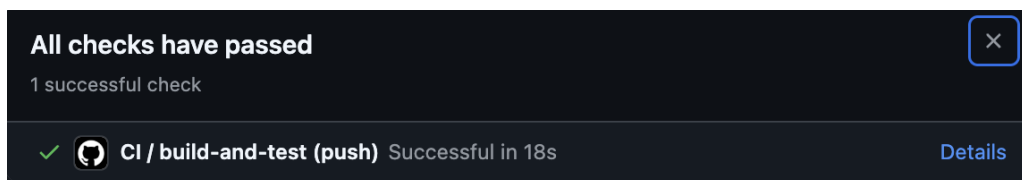
7 Continuous Integration

La continuous integration gira su GitHub Actions. Il senso è che a ogni push o pull request i test partono da soli, così se rompo qualcosa me ne accorgo subito invece che due settimane dopo.

Tutto sta nel file `.github/workflows/ci.yml`: fa la build Maven completa, lascia fuori i test taggati `ui` (che hanno bisogno di Chrome e dell'app avviata, e sul runner non ci sono) e alla fine salva il report JaCoCo come artifact scaricabile.

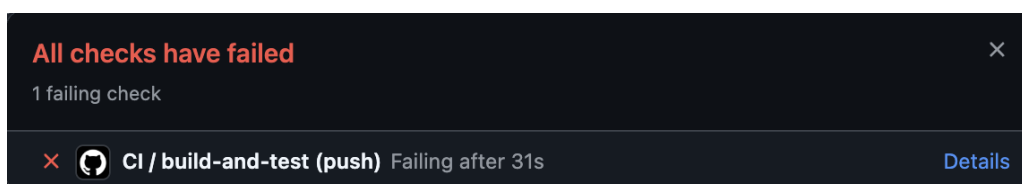
Listing 20: `.github/workflows/ci.yml` (estratto)

```
1 on:
2   push:
3   pull_request:
4
5 jobs:
6   build-and-test:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v4
10      - uses: actions/setup-java@v4
11        with:
12          distribution: temurin
13          java-version: '17'
14      - run: mvn -B -f java/pom.xml verify
15      - uses: actions/upload-artifact@v4
16        with:
17          name: jacoco-report
18          path: java/target/site/jacoco/
```

Figure 23: Esecuzione CI riuscita sul branch `main`

Questo è il run verde su `main`.

Solo che avere la pipeline verde, di per sé, dimostra poco: dimostra che i test passano, non che la CI se ne accorgerebbe se smettessero di passare. Per essere sicuro che il meccanismo funzionasse ho quindi rotto il codice apposta. Su un branch a parte, chiamato `ci_failure` e lasciato sul repository come prova, `assegnaPosto` è stato modificato per restituire sempre `TipoPosto.NESSUNO`. Al push la pipeline è andata in rosso, con i test di `ParcheggioTest` e `SmartParkingFSMTest` falliti sugli assert dell'assegnazione. Il branch `main` è rimasto com'era.

Figure 24: Esecuzione CI fallita sul branch `ci_failure` (regressione introdotta di proposito)

Quello che mi porto a casa da questa parte è che il valore della CI non è tanto far girare i test, che posso farlo anche in locale, quanto non potermene dimenticare: ogni commit resta legato a una prova che in quel momento il codice funzionava.

8 Conclusioni

Il progetto copre tutte e quattro le parti chieste dalla consegna: i requisiti, il modello Asmeta con gli scenari e il model checking, i contratti JML sul nucleo e l'implementazione Java con i test, la UI, la CI e l'analisi statica.

8.1 Riepilogo delle evidenze

- Simulatore e animatore Asmeta (Sezione 2)
- AsmetaV – copertura degli scenari Avalla (Sezione 2)
- AsmetaSMV – 8 CTLSPEC verificate e 2 volutamente false con controesempio (Sezione 2)
- Model Advisor e ATGT, opzionali (Sezione 2)
- OpenJML ESC sui contratti del nucleo (Sezione 3)
- UI Vaadin in esecuzione e test Selenium (Sezione 4)
- JUnit (59 test del package `core`, tutti verdi, più 1 test Selenium escluso dalla build standard), copertura JaCoCo e mutation testing con PIT, 90 mutanti su 90 uccisi (Sezione 5)
- Ispezione con checklist, seconda lettura con LLM e analisi statica con SpotBugs, da 7 bug a 0 (Sezione 6)
- GitHub Actions: build e test automatici a ogni push, più il branch `ci_failure` come prova che la pipeline intercetta le regressioni (Sezione 7)

8.2 Cosa mi porto a casa

La cosa che mi è rimasta di più, facendolo, è che le varie tecniche non si sovrappongono ma guardano posti diversi.

Il model checking mi ha detto che i contatori non possono impazzire in nessuno stato raggiungibile, cosa che con i test non avrei mai potuto dimostrare, al massimo controllare in qualche caso. L'ESC mi ha dato la stessa garanzia sul codice vero, però solo su una classe che ho dovuto tenere apposta piccola e senza dipendenze. I test coprono anche il resto, ma solo nei casi che mi sono venuti in mente, e infatti la coverage mi ha fatto scoprire due rami che mi ero dimenticato. L'ispezione, per ultima, ha trovato quello che tutte le altre si erano perse. Da un lato il difetto della memoria a slot singolo: il codice era conforme al modello, quindi nessuna verifica automatica poteva segnalarlo, perché il problema stava nel modello. Dall'altro, con SpotBugs, un problema di serializzazione della vista Vaadin che non avrei mai visto leggendo il codice, perché per accorgersene bisogna conoscere il contratto della libreria.

Se dovessi rifarlo, la cosa che cambierei è proprio quella: mettere l'associazione veicolo→posto già nell'ASM, invece di scoprirne la mancanza alla fine.